DQN in Bits Sequence Generation Task

Name: Zijian Zhao

University: School of Computer Science and Technology, Sun Yat-sen University

E-mail: zhaozj28@mail2.sysu.edu.cn

Phone: 18345167686

Please refer to the Readme file in the code directory for instructions on how to run the code.

If you find the report to be excessively lengthy, you can refer to Section 7 (Supplement), which provides concise answers to the questions outlined in your problem description.

1 Abstract

This project focuses on the application of the Deep Q-Network (DQN) method in the task of bit sequence generation. Firstly, a DQN structure is designed, which includes the reward function, network architecture, and action space. Several improvements are made to the original DQN algorithm.

In order to address the issue of sparse rewards, alternative methods such as Hindsight Experience Replay (HER) and Curriculum Learning are explored. However, during the experiments, it was observed that Curriculum Learning in this task may lead to catastrophic forgetting or deplete the model's capacity. To overcome this problem, **a novel mechanism is proposed that involves stepwise unfreezing of network parameters in curriculum learning**. The experiments have demonstrated that **this method outperforms all others mentioned in this report**, yielding the best performance.

2 Problem Definition

In this section, I will define the basic elements of the bit sequence generation task. Similar to the bit flipping task, in each episode, a target sequence is given, and the goal is to make the generated state sequence match the target sequence. However, I consider the bit sequence generation task to be more challenging due to its larger observation space, as the length of the state sequence varies during generation.

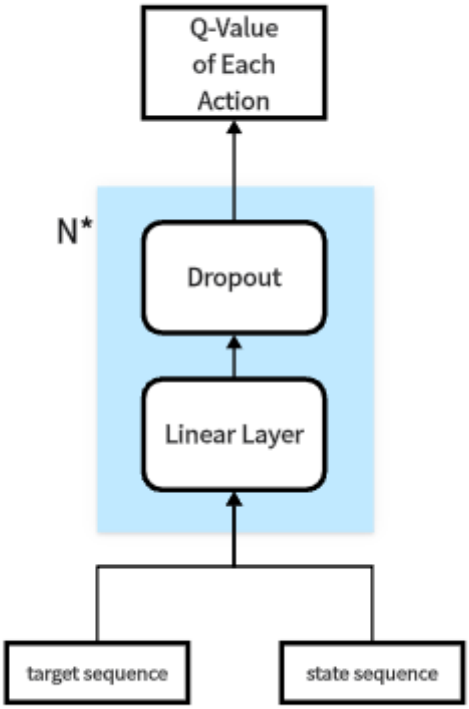
In Reinforcement Learning (RL), there are five main components: the environment, policy, observation (state), action, and reward. Initially, I will define a simple version, and the modifications to it will be discussed in Section 3.2. In this experiment, the observation consists of the target sequence and the current state sequence (i.e., the sequence that has been generated). The action, denoted as $a \in \{0, 1\}$, represents the token that will be generated in the next step. Specifically, the observation space is defined as $o = [o_t, o_s] \in \{0, 1, -1\}^{2n}$, where $o_t \in \{0, 1\}^n$ is the target sequence and $o_s \in \{0, 1, -1\}^n$ represents the state sequence (-1 indicates that the token has not been generated). Here, n is the length of the target sequence. The reward function

is a sparse version defined as $R := (o_t == o_s)$. It is evident that the reward is only 1 when the sequence is completely generated; otherwise, it is always 0. The performance of various shaped reward functions will also be explored in Section 3.2.1.

3 Original DQN

3.1 Network Architecture

In this section, I employ a simple three-layer Feedforward Network (FFN) as the policy network. After each linear layer, I include an additional Dropout layer, which demonstrates improved performance as discussed in Section 4.2.1. The output dimension of the network is 2, representing the corresponding Q-values of the two actions (0 and 1, i.e., the next generated token).



3.2 Further Studies on DQN

Based on the conducted experiments, it was observed that the original DQN algorithm exhibited poor performance in this task, struggling to accurately generate sequences exceeding 10 tokens. In order to enhance the model's performance, several modifications were attempted by altering key components of the RL framework. However, it should be noted that not all of these methods proved to be effective. To compare the effectiveness of these methods, I primarily tested their performance on n=5 and evaluated the maximum length of sequences they could accurately generate within 10,000 episodes. (Ideally, a larger number of episodes would be more suitable as most RL methods converge over millions of iterations. However, due to time constraints, I limited the comparison to 10,000 episodes.)

3.2.1 Reward Function

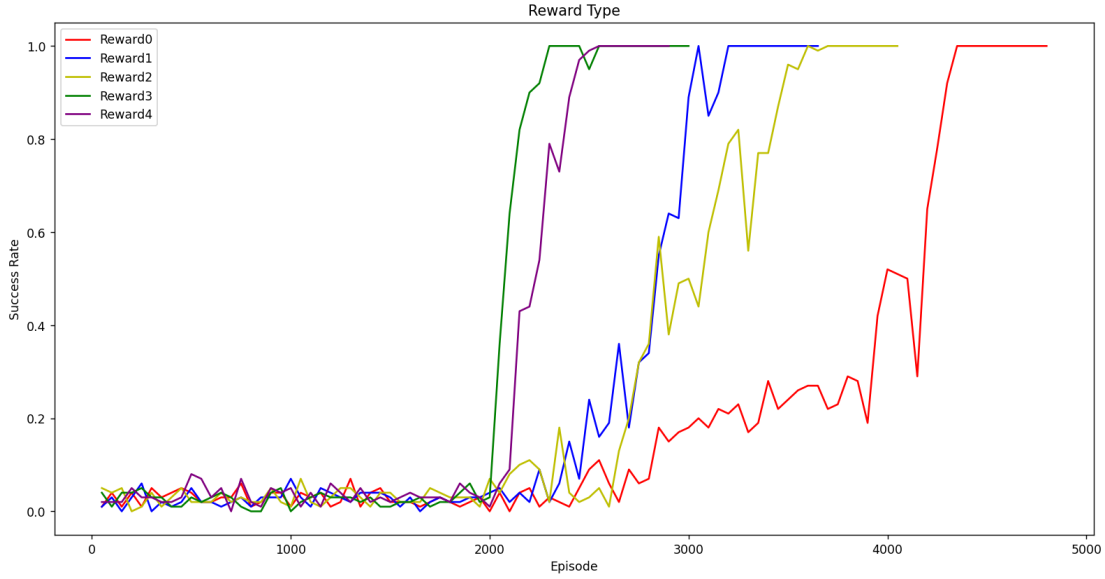
Shaped rewards are effective in reducing unnecessary search space. To investigate the impact of different reward functions, I designed several reward functions as follows. The original reward function, denoted as Reward Function 0, is defined as $R_0(i) := (o_t == o_s)$.

Reward Function1: $R_1(i) := (i == n) * similarity(o_t, o_s)$
Reward Function2: $R_2(i) := similarity(o_t, o_s)$
Reward Function3: $R_3(i) := \frac{o_t[i]==o_s[i]}{n}$
Reward Function4: $R_4(i) := \begin{cases} -1 & o_t[1:i-1] == o_s[1:i-1] \ \& \ o_t[i] \neq o_s[i] \\ (o_t == o_s) & otherwise \end{cases}$

where i is the step, $x[a:b]$ means $\{x[a], x[a+1], \ldots, x[b-1], x[b]\}$ and x means $x[0:n]$, n is the length of vector x.

Similar to R_0, R_1 is also a sparse reward but provides more information. In this experiment, I define $similarity(x, y) := \frac{\sum_{i=1}^{n}(x[i]==y[i])}{n}$, where n is the length of x or y. Both R_2 and R_3 allow for backward propagation at each step, but R_3 only considers the current state change. R_4, R_5 are modification of R_0, R_3, where the agent receives a negative reward when generating the wrong token for the first time in each episode. This modification helps the agent avoid making early mistakes.

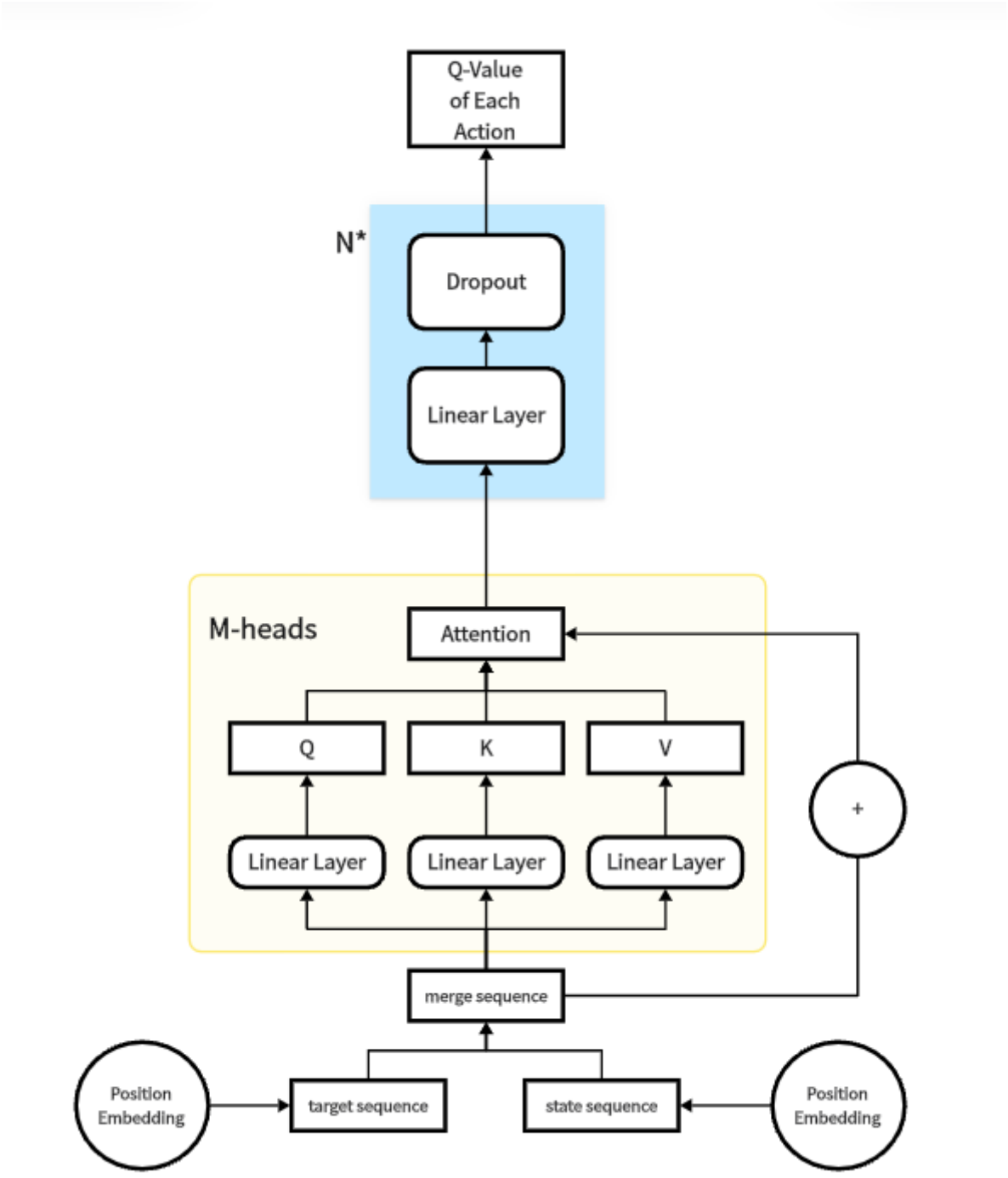
Reward Function	Maximum Length of Sequence (could accurately generate within 10,000 episodes)
R_0	6 (within 6400 episodes)
R_1	9 (within 9900 episodes)
R_2	8 (within 6900 episodes)
R_3	**19~21 (The model tends to converge around 9000 episodes, despite experiencing notable fluctuations along the way.)**
R_4	**16 (within 9650 episodes)**



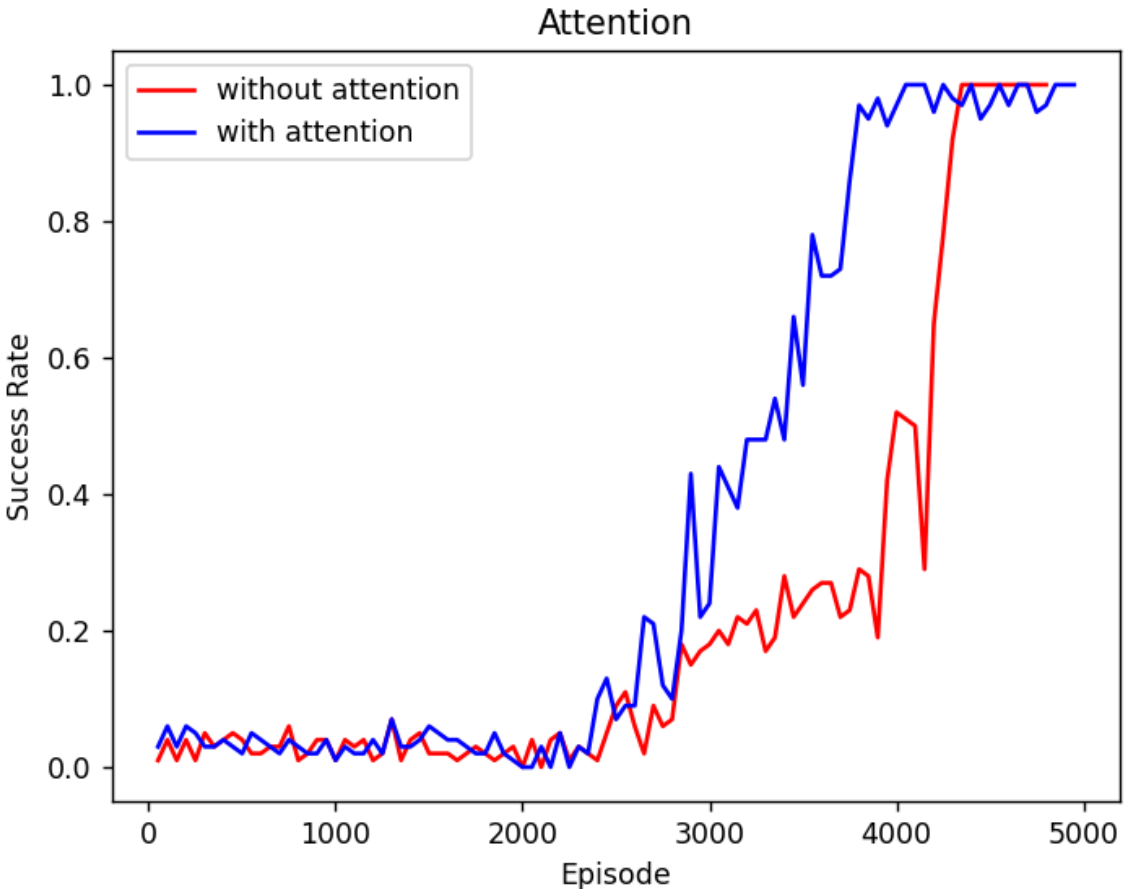
Through experimentation, it has been observed that R_3 exhibits the best performance due to its complete reward. Additionally, although R_4 also utilizes a sparse reward mechanism, it enhances the model by incorporating penalties for initial mistakes.

3.2.2 Network Architecture

Considering that this task involves generating the target token corresponding to the position of the first "-1" token in the state sequence, I believe that incorporating an attention mechanism to capture contextual information could be beneficial. Consequently, I included an additional attention layer before the linear layers. However, it appears that this modification did not yield the desired results and may have even negatively affected the performance of the model. Currently, I do not have a clear understanding of the underlying reasons for this outcome, but I am eager to further explore and study this matter in the future.

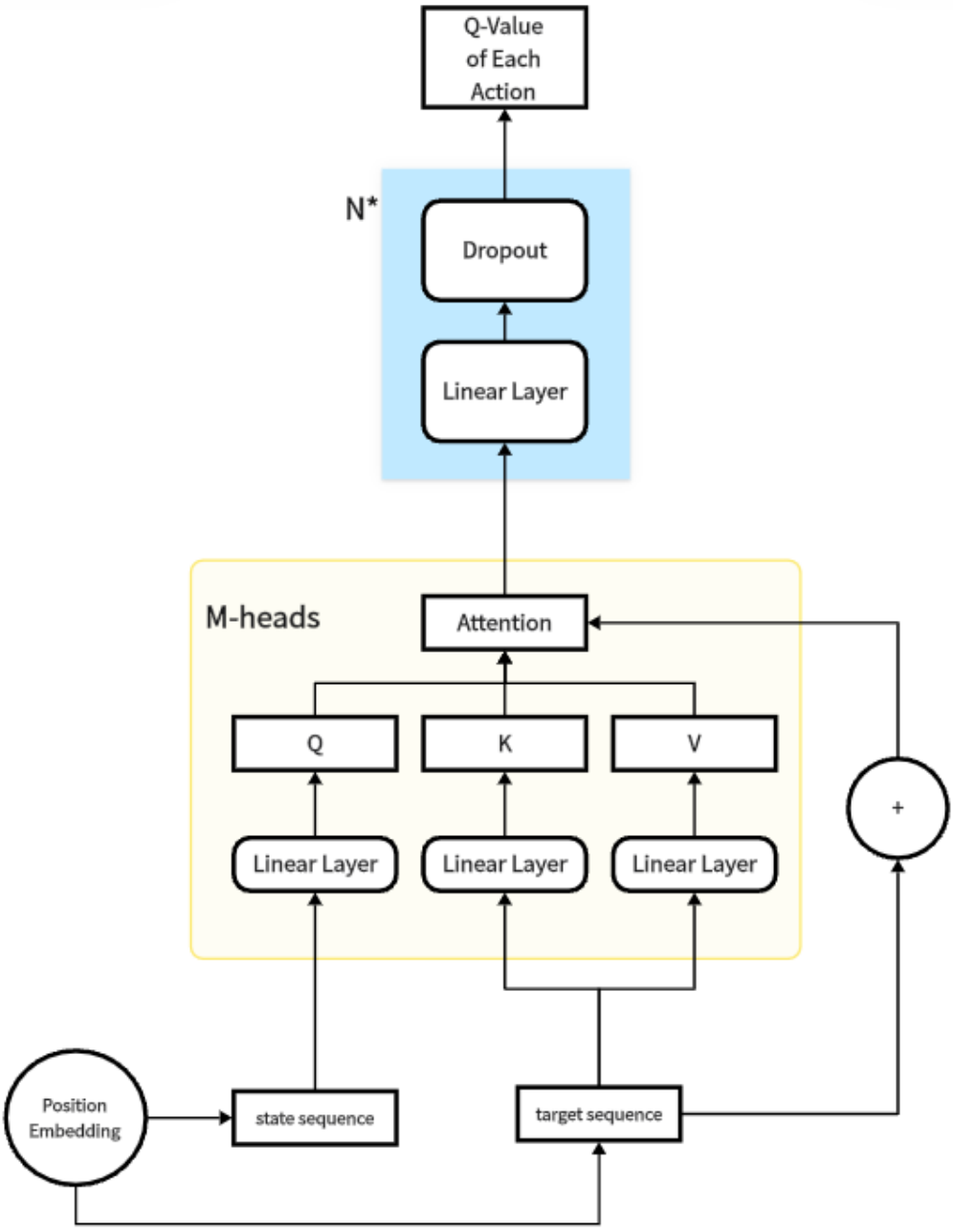


Attention	Maximum Length of Sequence (could accurately generate within 10,000 episodes)
w/o	**6 (within 6400 episodes)**
w	5 (within 4550 episodes) (need about 11000 episodes to converge in length=6)



In the two comparative experiments, it is evident that the attention mechanism can facilitate faster convergence of the model. However, it may also introduce a higher degree of fluency-related issues when the model has already achieved a relatively high level of accuracy.

One possible improvement is to use the state sequence as the query and the target sequence as the key and value. I believe that the reason the current attention-based network is not performing well is because it tries to capture the internal relationships within the target and state sequences. However, since the tokens are actually generated randomly, there are no inherent relationships between them. Unfortunately, due to time constraints, I have not yet completed the experiment for the new version.



3.2.3 Observation (State) & Action Space

In this experiment, the agent generates tokens one by one in each episode. Let's imagine that we view the target sequence as a whole using decimal representation. We can then train the network to generate the entire state sequence, which is also represented in decimal form, in just one step. This is obviously a simpler task for the network. However, the practical application of this approach is limited. For example, in tasks involving object manipulation with a robotic arm, as mentioned in [2], it is not feasible to generate the entire action sequence all at once. Nevertheless, drawing inspiration from this idea, I believe it is worth exploring the possibility of merging neighboring states and generating sequential tokens simultaneously.

group	Maximum Length of Sequence (could accurately generate within 10,000 episodes)
1	**6 (within 6400 episodes)**
2	4 (within 5550 episodes)
3	3 (within 4950 episodes)

However, the experimental results differ from the original hypothesis, which suggested that as the group size increases, the model becomes more difficult to converge. For instance, when the group size is set to 2, the model achieves only about 10% accuracy with a sequence length of 6. I believe that the reason for the underperformance of the model compared to the initial speculation lies in the fact that the network's output is in the form of Q-values, which differs from directly outputting actions. Therefore, it is worth exploring in future research whether this grouping mechanism is beneficial in the context of Policy Gradient as well.

4 Improved Version

To address the issue of sparse rewards, in this section, I solely utilize the R_0 reward function.

4.1 Hindsight Experience Replay (HER)

To address the challenge of sparse rewards, Andrychowicz et al. introduced the concept of Hindsight Experience Replay (HER). In HER, during experience replay, a new target is assigned to past experiences by sampling from the next_state buffer. This technique increases the number of non-zero reward samples in the buffer, facilitating faster learning of effective strategies by the agent.

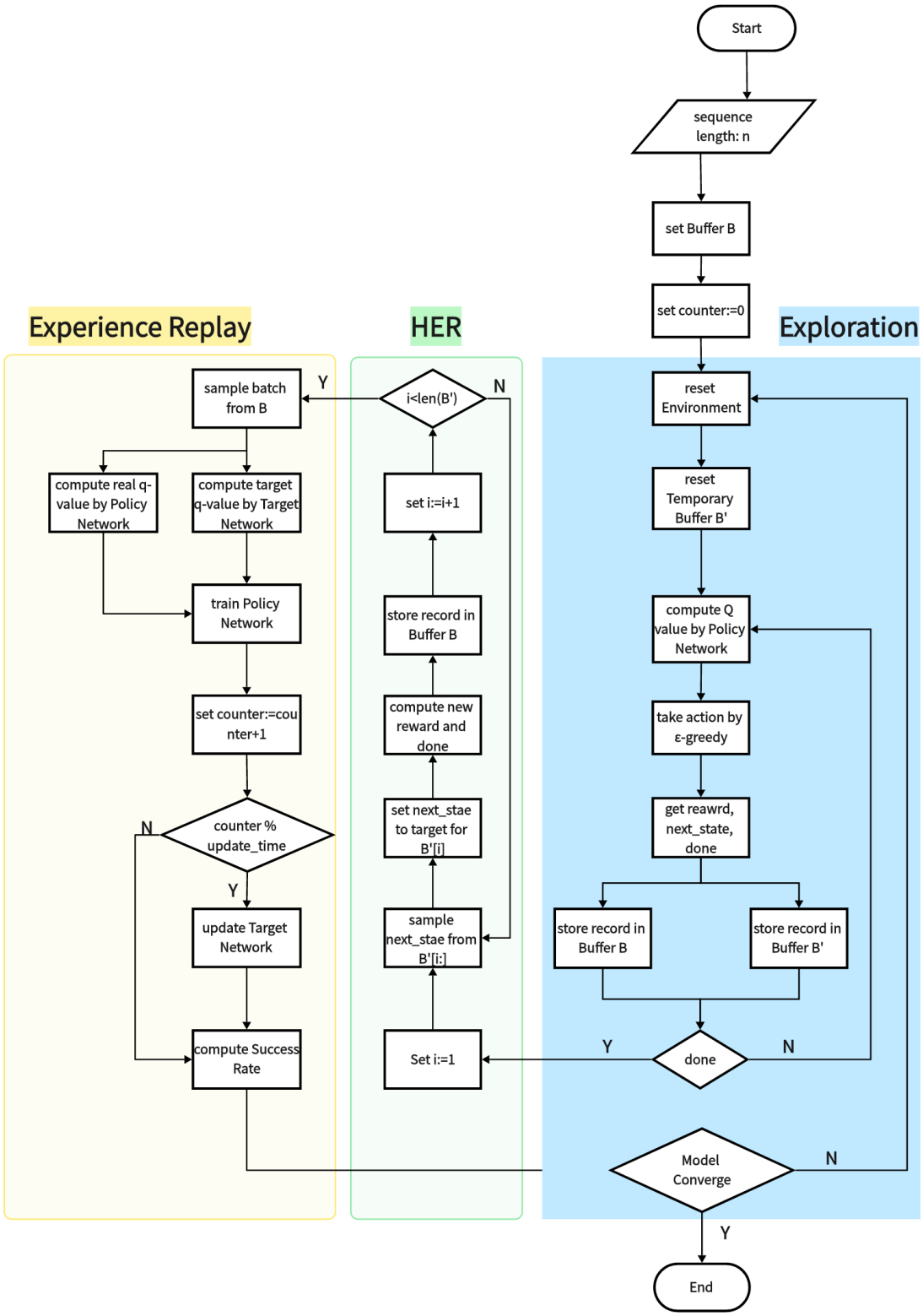
Algorithm 1 Hindsight Experience Replay (HER)

Given:

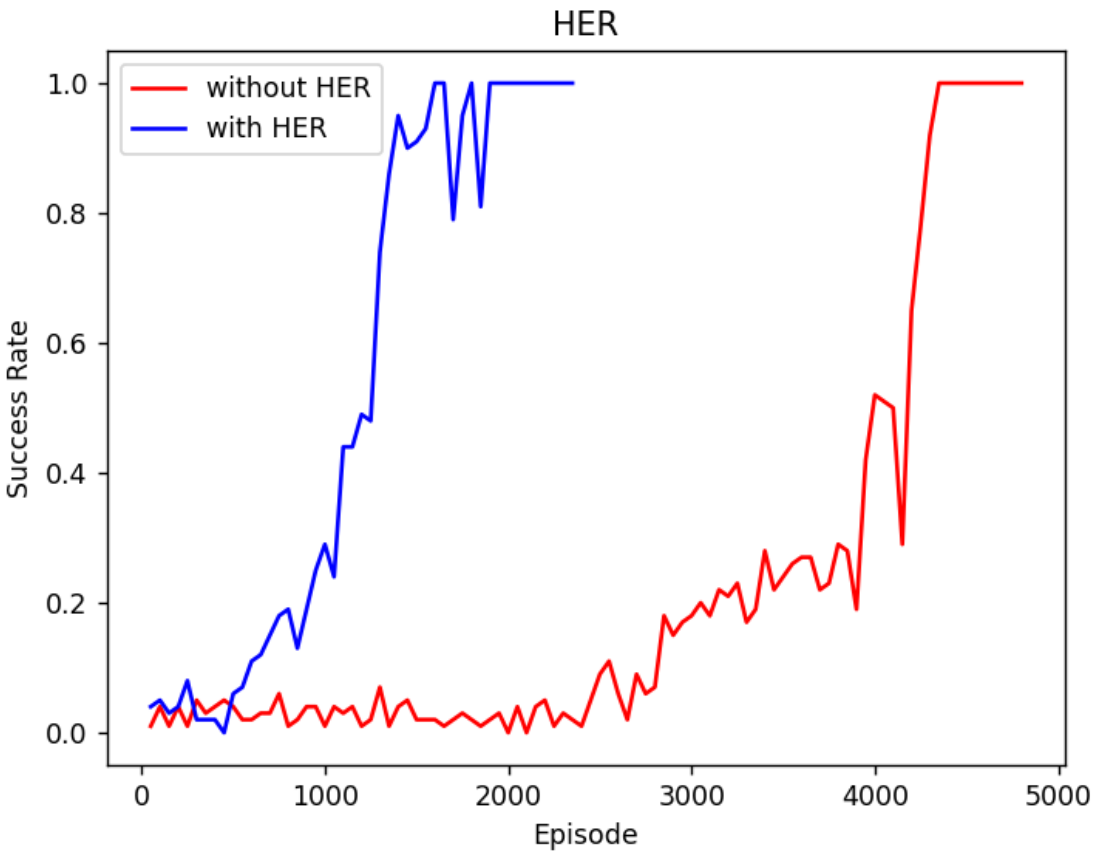
- an off-policy RL algorithm $\mathbb{A}$, ▷ e.g. DQN, DDPG, NAF, SDQN
- a strategy $\mathbb{S}$ for sampling goals for replay, ▷ e.g. $\mathbb{S}(s_0, \ldots, s_T) = m(s_T)$
- a reward function $r : \mathcal{S} \times \mathcal{A} \times \mathcal{G} \to \mathbb{R}$. ▷ e.g. $r(s, a, g) = -[f_g(s) = 0]$

Initialize $\mathbb{A}$ ▷ e.g. initialize neural networks
Initialize replay buffer R
for episode = $1, M$ **do**
  Sample a goal g and an initial state s_0.
  for $t = 0, T-1$ **do**
    Sample an action a_t using the behavioral policy from $\mathbb{A}$:
      $a_t \leftarrow \pi_b(s_t||g)$ ▷ $||$ denotes concatenation
    Execute the action a_t and observe a new state s_{t+1}
  end for
  for $t = 0, T-1$ **do**
    $r_t := r(s_t, a_t, g)$
    Store the transition $(s_t||g,\ a_t,\ r_t,\ s_{t+1}||g)$ in R ▷ standard experience replay
    Sample a set of additional goals for replay $G := \mathbb{S}$(**current episode**)
    for $g' \in G$ **do**
      $r' := r(s_t, a_t, g')$
      Store the transition $(s_t||g',\ a_t,\ r',\ s_{t+1}||g')$ in R ▷ HER
    end for
  end for
  for $t = 1,\ N$ **do**
    Sample a minibatch B from the replay buffer R
    Perform one step of optimization using $\mathbb{A}$ and minibatch B
  end for
end for

The algorithm for HER, as presented in [2], is depicted in the above image. However, the yellow section is not very clear. Therefore, I have created a flowchart based on the information obtained from the website.



HER	**Maximum Length of Sequence (could accurately generate within 10,000 episodes)**
w/o	6 (within 6400 episodes)
w	**10 (within 8950 episodes)**



The experiments demonstrate that HER exhibits significant improvements over the original DQN.

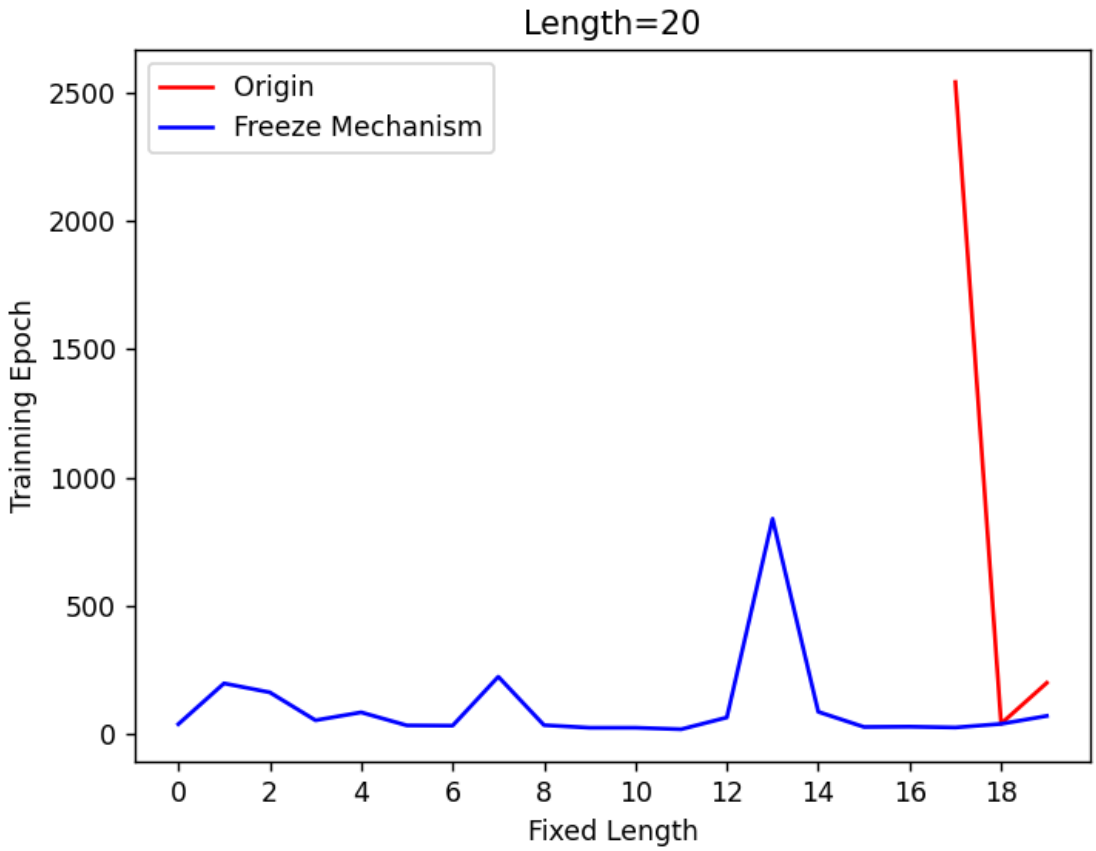
4.2 Curriculum Learning

Curriculum Learning is an efficient method for networks to learn difficult tasks. It involves initially exposing the network to easier tasks and gradually increasing the difficulty. In a study by [3], they implemented Curriculum Learning by adjusting the distribution of different datasets with different tasks. In the field of reinforcement learning, there is also relevant research, particularly in robotics. For instance, [4] applied Curriculum Learning to a task where a robot needs to move to a target point. This task is similar to ours as it also involves sparse rewards. In the initial episodes, the robot is initialized with a starting point that is relatively close to the target, making it easier to obtain a non-zero reward. As the training progresses, the initial point is gradually moved farther away from the target point. Inspired by this mechanism, I have designed a Curriculum Learning method for the Bits Generation Task.

4.2.1 Naive Method

In the Bits Generation Task, the generation of sequences of different lengths corresponds to curriculum with varying levels of difficulty. Initially, I designed a naive mechanism where a fixed length, denoted as l, was set such that the first l bits of the state sequence matched the target sequence. This approach effectively reduced the task difficulty by increasing the agent's chances of obtaining a non-zero reward. As the model converged, l would gradually decrease until it reached 0.

However, through experimentation, it was observed that as l decreased, the model required more episodes to converge. In my experiment, I set $n = 20$ and employed a three-layer FFN to learn the task. Unfortunately, the model failed or became hard to converge when l decreased to 16 or 17. I believe that a possible reason for this is the limited capacity of the model, as mentioned in [4]. When the model is saturated, it becomes challenging to acquire new knowledge, posing a challenge in continuous learning and federated learning scenarios. To address this issue, I propose a novel mechanism involving the stepwise unfreezing of network parameters in curriculum learning, which will be discussed in the next subsection.

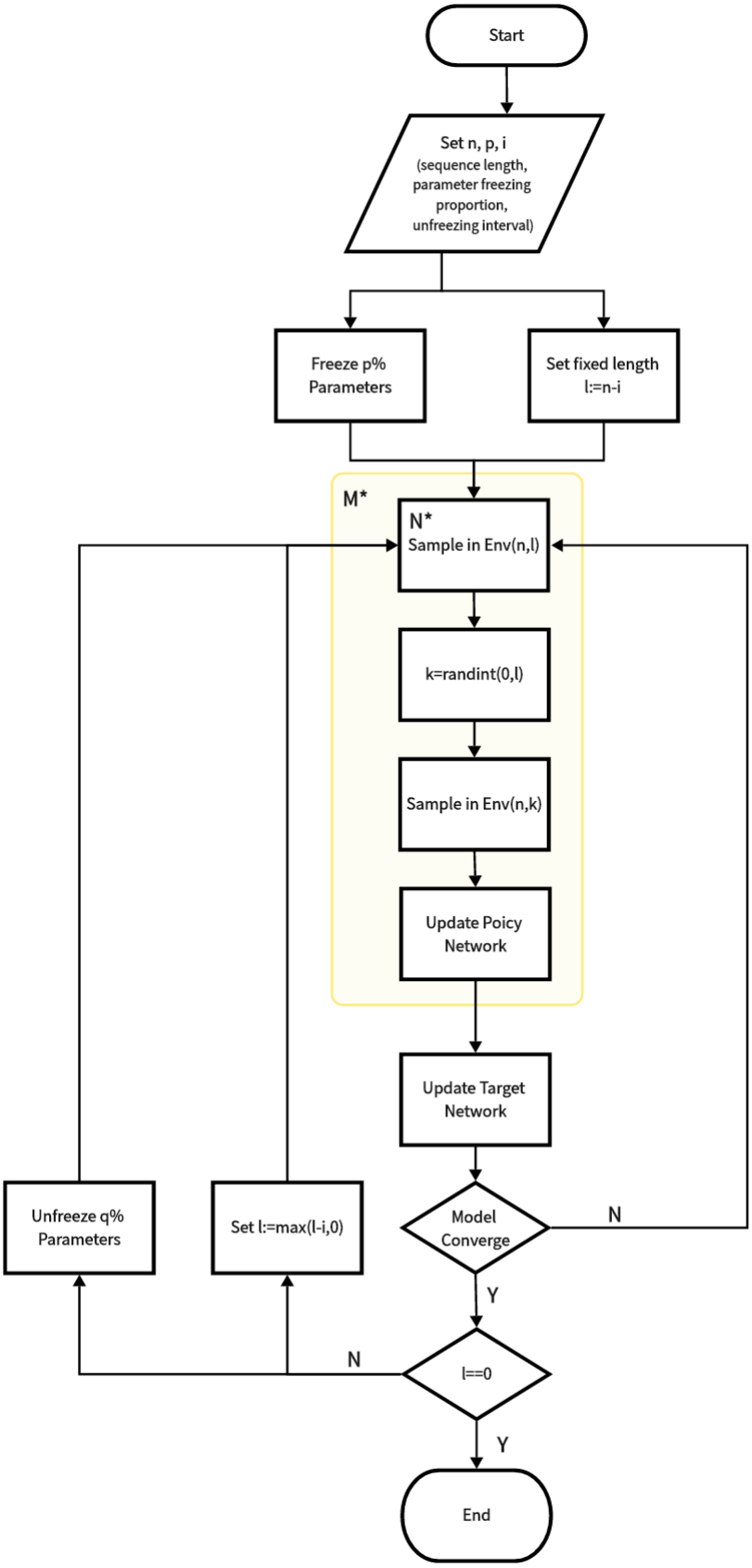


Please observe this picture from right to left. You will notice that the original Curriculum Learning method failed to converge when the fixed length was set to 17, so it lacked the left part data. However, the new method discussed in the following subsection successfully addresses this issue. (1 epoch = 50 episodes)

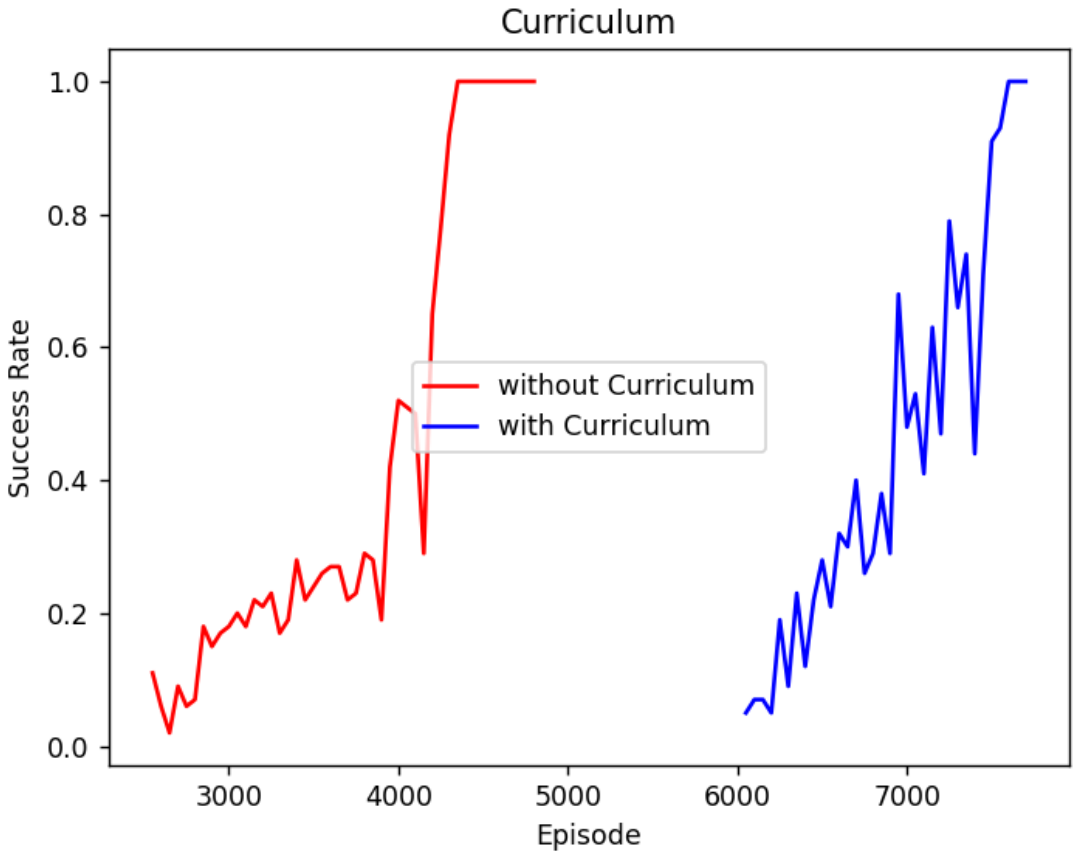
4.2.2 Proposed Method

To address the issue of limited model capacity, I have designed an unfreezing mechanism that initially freezes certain parameters. As the fixed length decreases, these parameters are progressively unfrozen.

Furthermore, drawing inspiration from the aggressive mechanism described in [5], I have incorporated a similar approach. As mentioned in the preceding subsection, a fixed length is assigned for each episode. However, I acknowledge that this fixed length may lead to overfitting. To mitigate this, I introduce a randomization component to the mechanism. After a specified number of episodes, the fixed length is set to a randomly chosen value. This variation enables the model to learn from a more diverse and representative environment.



In the flowchart, the term "Env(a,b)" denotes a sequence with a length of 'a', where the first 'b' bits of both the target sequence and the state sequence are identical.



The figure illustrates the training process of DQN with and without Curriculum Learning. However, it should be noted that the Curriculum Learning DQN may require more initial episodes to gather sufficient experience. To ensure fairness, only the episodes during model training are displayed. It is evident that the Curriculum Learning DQN is capable of converging in a shorter time compared to the standard DQN.

Through experimentation, my proposed method successfully solves the bit generation task with a sequence length of 35. It outperforms the other models mentioned in this paper. However, it was observed that the method fails to solve the problem when the sequence length is increased to 50. (Based on the experimental findings, it appears that employing a deeper network may be beneficial.) In the next section, a potential improvement is proposed to address this limitation. Due to time constraints, I did not explore whether the method can solve tasks with sequence lengths ranging from 36 to 49.

5 Conclusion

In this project, I **propose a curriculum learning method with stepwise unfreezing of parameters** for the task of bit sequence generation. The unfreezing mechanism helps address the issues of limited model capacity and catastrophic forgetting. It has been successfully demonstrated to generate sequences of length 35 using sparse rewards. However, it has been observed that the model struggles to converge when the sequence length is larger, such as 50. Therefore, **in future work**, I suggest exploring an **adaptive unfreezing mechanism** where, if the model fails to converge after a certain number of episodes, specific parameters can be unfrozen dynamically.

During the experiments, it was observed that the attention mechanism did not prove to be effective in this particular task, which is somewhat surprising considering that attention layers are generally known to enhance the learning of contextual relationships. In our task, which involves mapping numbers in the target sequence to their corresponding positions in the state sequence, the lack of improvement from the attention mechanism deserves further investigation.

Furthermore, this project has provided valuable insights for my ongoing research. Specifically, I am currently focused on using WiFi signals to assist a small robot rat equipped with IMU sensors in navigating obstacles. Existing WiFi sensing methods have limitations in terms of transferability and robustness, and the majority of research in the field of WiFi in robotics revolves around localization. Inspired by the bit sequence generation task in this project, I believe that training the robot to move towards a target point instead of crossing obstacles using WiFi signals could help mitigate the issue of drift when relying solely on IMU sensors. This problem shares similarities with the challenges posed by sparse rewards and action sequences encountered in the bit sequence generation task.

6 References

[1] Mnih, V., Kavukcuoglu, K., Silver, D. *et al.* Human-level control through deep reinforcement learning. *Nature* 518, 529–533 (2015).

[2] Andrychowicz M, Wolski F, Ray A, et al. Hindsight experience replay[J]. Advances in neural information processing systems, 2017, 30.

[3] Bengio Y, Louradour J, Collobert R, et al. Curriculum learning[C]//Proceedings of the 26th annual international conference on machine learning. 2009: 41-48.

[4] Florensa C, Held D, Wulfmeier M, et al. Reverse curriculum generation for reinforcement learning[C]//Conference on robot learning. PMLR, 2017: 482-495.

[5] Wu L, Guo S, Wang J, et al. On Knowledge Editing in Federated Learning: Perspectives, Challenges, and Future Directions[J]. arXiv preprint arXiv:2306.01431, 2023.

[6] Chen C, Xu H, Wang W, et al. Communication-efficient federated learning with adaptive parameter freezing[C]//2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS). IEEE, 2021: 1-11.

7 Supplement

1. How do you design the network architecture?

   You can refer to the network structure diagram (a simple FFN) in Section 3.1. Additionally, I also implemented an attention-based version (by adding an attention layer before the linear layer) as described in Section 3.2.2. However, I observed that its performance was not very satisfactory. Consequently, I plan to make some modifications to the current attention-based version, which are discussed at the end of Section 3.2.2.

2. How do you design the output space for the DQN? Should it generate the sequence token by token or in other ways?

   The output dimension of the DQN is 2, representing the q-values of two actions. In Section 3.2.3, I compared the performance of the DQN when generating tokens one by one with that of generating multiple tokens at once.

3. Does the algorithm work? How does the success rate change with varying lengths?

   The original DQN algorithm seems to only work well for short sequence lengths. As the length of the sequence increases, the number of training episodes required also increases significantly. However, in Section 3.2.1, I explored the use of shaping reward functions, which proved to be helpful in addressing this problem.

4. If it works for small values of , why doesn't it work for larger values (where the space is much larger)? What potential challenges do you think there are?

   I believe there are two main challenges. Firstly, as the sequence length increases, the exploration space grows larger. This becomes especially challenging in scenarios with sparse rewards, where finding a non-zero reward becomes more difficult. Secondly, for larger input lengths, even training the network to directly generate ground-truth tokens becomes more challenging.

5. How does the improved version of the algorithm compare to the original version?

   There are three improved versions that appear to work well, including a reward function R_3 mentioned in Section 3.2.1, the HER algorithm, and my method based on Curriculum Learning. I have compared their performance in the table below.

Model	**Length of Sequence (that model can generate correctly)**
Original DQN	6 (within 6400 episodes) (If the length is set to 7, even after 50000 episodes, the model only achieves a success rate of approximately 5%. Interestingly, it appears that the success rate plateaus around 35000 episodes, showing no significant improvement thereafter.)
R_3 Reward Function	25 (within 57100 episodes) (If the length is longer, the success rate exhibits significant fluctuations during training.)
HER	17 (within 109200 episodes)
My method (based on Curriculum Learning)	35 (within 187700 episodes)

   Due to time constraints, I was not able to conduct a sufficient number of experiments. While the models demonstrated the ability to solve the given problems, it does not necessarily imply that they are incapable of addressing more challenging tasks with longer sequence lengths.

6. Visualization: Provide visualizations of the training process and results, such as the success rate for reaching the goals and how it changes with varying lengths, the targets and the generated sequences.

   Please refer to the experimental result images included in the main text.

7. Challenges: Discuss any challenges you faced during the implmentation and evaluation process, and how you overcame them.

One of the major challenges I faced was the ineffectiveness of the curriculum learning method mentioned in Section 4.2.1 when dealing with sequences of significant length. This led me to recall previous papers that highlighted the importance of considering model capacity and catastrophic forgetting in continuous learning scenarios. Consequently, I devised a new method, as detailed in Section 4.2.2, which addressed this issue by gradually unfreezing model parameters.

8. Improvements: Provide a detailed explanation of how the improved version in (3) works and how it improved the success rate of the DQN algorithm.

   I have proposed a novel mechanism in Section 4.2.2 that involves the stepwise unfreezing of network parameters in curriculum learning. This mechanism aims to enhance the learning process by gradually allowing the network to adapt to more complex patterns and representations. Please refer to Section 4.2.2 for a detailed explanation of this proposed mechanism.

9. Conclusion: Summarize your findings and discuss what you think about for this problem.

   Please refer to Section 5.

8 Appendix —— Details in Code

8.1 Network

```
class DQN(nn.Module):
    def __init__(self, state_dim, action_dim=2, hidden_dims=[256,256,256], dropout_rate=0.3):
        super().__init__()
        self.model=nn.ModuleList()
        self.model.append(nn.Linear(state_dim,hidden_dims[0]))
        for i in range(1,len(hidden_dims)):
            self.model.append(nn.Linear(hidden_dims[i-1], hidden_dims[i]))
        self.model.append(nn.Linear(hidden_dims[-1],action_dim))
        self.relu=nn.ReLU()
        self.drop=nn.Dropout(dropout_rate)
        self.sigmoid=nn.Sigmoid()

    def forward(self,x):
        x=x.to(torch.float32)
        for layer in self.model[:-1]:
            x=layer(x)
            x=self.relu(x)
            x=self.drop(x)
        x=self.model[-1](x)
        #x=self.sigmoid(x)
        return x
```

The original DQN is a straightforward FFN that allows you to specify the hidden layers and dimensions using the "hidden_dims" parameter.

```
class DQN_Attention(nn.Module):
    def __init__(self, state_dim, action_dim=2, hidden_dims=[256,256,256], num_heads=2, emb_dims=4, dropout_rate=0.3, device=torch.device("cpu")):
        super().__init__()
```

```
        self.state_dim=state_dim
        self.emb_dims=emb_dims
        self.emb=nn.Embedding(3,emb_dims)

 self.attention=nn.MultiheadAttention(embed_dim=emb_dims,num_heads=num_heads
,dropout=dropout_rate)
        self.Q = nn.Linear(emb_dims, emb_dims)
        self.K = nn.Linear(emb_dims, emb_dims)
        self.V = nn.Linear(emb_dims, emb_dims)
        self.model=nn.ModuleList()
        self.model.append(nn.Linear(emb_dims*state_dim,hidden_dims[0]))
        for i in range(1,len(hidden_dims)):
            self.model.append(nn.Linear(hidden_dims[i-1], hidden_dims[i]))
        self.model.append(nn.Linear(hidden_dims[-1],action_dim))
        self.relu=nn.ReLU()
        self.drop=nn.Dropout(dropout_rate)
        self.sigmoid=nn.Sigmoid()
        self.pos_emb=self.positional_encoding(state_dim//2,emb_dims)

 self.pos_emb=torch.cat([self.pos_emb,self.pos_emb],dim=-2).to(device)

    def forward(self,x):
        x[x==-1]=torch.max(x)+1
        x = x.to(torch.long)
        if len(x.shape)==1:
            x=x.view(1,-1)
        x=self.emb(x)
        y=x+self.pos_emb
        y,_=self.attention(self.Q(y),self.K(y),self.V(y))
        y+=x
        x=x.view(-1,self.state_dim*self.emb_dims)
        for layer in self.model[:-1]:
            x=layer(x)
            x=self.relu(x)
            x=self.drop(x)
        x=self.model[-1](x)
        #x=self.sigmoid(x)
        return x.squeeze()
```

In the attention-based version, an attention layer is introduced before the linear layers. Specifically, positional embedding is applied to the target and state sequences separately. Afterwards, three linear layers are employed to generate the key, query, and value, respectively.

8.2 Environment

The environment, which inherits from the gym.Env class, is presented as follows.

```
# Bits Generation Environment
class Bits_Env(gym.Env):
    '''
    Input:
        n: length of sequence
        reward_type: refer to README for details
```

```
        group: number of tokens as a whole
    '''
    def __init__(self,n,reward_type=0,group=1):
        super().__init__()
        self.len=(n//group)*group # length must be divided exactly by group
        self.target=np.random.randint(2**group,size=self.len)
        self.pad=-1 # token has not been generated
        self.state=np.array([self.pad]*self.len)
        self.reward_type=reward_type
        self.pos=0 # the position where to generate token in next step

    '''
    Description:
        reset the whole environment including 'state' and 'target'
    Input:
        n: the begining position of state sequence  (i.e.
state[:n]=target[:n])
        seed & options: parameters of 'reset' function in gym.Env
    Output:
        a dictionary including 'state' and 'target'
    '''
    def reset(self,n=0,seed=None,options= None):
        super().reset(seed=seed,options=options)
        self.pos=n
        self.target=np.random.randint(2,size=self.len)
        self.state=self.target.copy()
        self.state[self.pos:]=self.pad
        return
{'state':torch.tensor(self.state),'target':torch.tensor(self.target)}

    '''
    Description:
        reset state sequence
    Input:
        n: the begining position of state sequence  (i.e.
state[:n]=target[:n])
    Output:
        a dictionary including 'state' and 'target'
    '''
    def reset_state(self,n=0):
        self.state=self.target.copy()
        self.pos=n
        self.state[self.pos:]=self.pad
        return
{'state':torch.tensor(self.state),'target':torch.tensor(self.target)}
```

In the reset function, the target is randomly generated, and the state is initialized. For curriculum learning, the state can be reset individually, and you have the option to set the starting point. Prior to the starting point, the state sequence is identical to the target sequence.

```
'''
Input:
    action: the generated token in current step
Output:
    a dictionary including 'state' and 'target', reward, and is_done
```

```
'''
def step(self, action):
    self.state[self.pos]=action
    self.pos+=1
    reward=self.get_reward()
    terminated=(self.pos==self.len)
    return
{'state':torch.tensor(self.state),'target':torch.tensor(self.target)},reward
,terminated

def get_reward(self):
    return self._compute_reward(self.target,self.state,self.pos-1)

'''
Input:
    target: target sequence
    state: state sequence
    action: the generated token in current step
Output:
    reward
'''
def compute_reward(self,target,state,action):
    pos=torch.where(state==self.pad)[0][0].item()
    state=state.clone()
    state[pos]=action
    return self._compute_reward(target,state,pos),(state==target).all()

'''
Input:
    target: target sequence
    state: state sequence
    pos: the position of the token generated in the last step
Output:
    reward
'''
def _compute_reward(self,target,state,pos=0):
    if self.reward_type==0:
        return float((state==target).all())
    elif self.reward_type==1:
        if state[-1]==self.pad:
            return 0
        else:
            return torch.sum(torch.tensor(state==target)).item()/self.len
    elif self.reward_type==2:
        return torch.sum(torch.tensor(state == target)).item() / self.len
    elif self.reward_type==3:
        return float(target[pos]==state[pos])/self.len
    elif self.reward_type==4:
        if (self.target[:pos]==self.state[:pos]).all():
            if self.target[pos]==self.state[pos]:
                return float((state == target).all())
            else:
                return -1
        else:
            return 0
```

The reward function implements the four types of rewards mentioned in Section 3.2.1 individually.

8.3 DQN Agent

8.3.1 Original Agent

```
# select action by ϵ-greedy
def select_action(self,eps,state,target):
    rand=torch.rand(1).item()
    if rand<eps: # choose action randomly
        return math.ceil(rand/(eps/(2**self.env.group)))
    else: # choose action with the max Q-value
        return
self.policy_network(torch.cat([state,target],dim=-1).to(self.device)).max(dim
=-1)[1].item()
```

In the DQN algorithm, the action is selected using an ε-greedy strategy, where ε denotes the exploration rate. The value of ε decreases gradually over time.

```
while not done:
    self.policy_network.train()
    state=obs['state'].to(self.device)
    target=obs['target'].to(self.device)
    action=self.select_action(self.eps,state,target)
    obs,reward,done=self.env.step(action)
    # store elements

 self.replay_buffer.store({'state':state.to(self.device),'target':target.to(
self.device),'reward':reward,'next_state':obs['state'].to(self.device),'acti
on':action,'done':done})
    state_episode[index]=state.to(self.device)
    next_state_episode[index]=obs['state'].to(self.device)
    reward_episode[index]=reward
    done_episode[index]=done
    action_episode[index]=action
    index+=1
```

In each episode, the data will be stored in a buffer for replay, which is used to train the network.

```
# only when the buffer is full, the network will train
if self.replay_buffer.is_full():
    loss.append(self.update_policy())
if i % update_episode==0:
    self.update_target()
    # evaluate current model
    success_rate=0
    for k in range(100):
        success_rate+=self.eval()
    success_rate/=100
```

When the buffer is full, at the end of each episode, the policy network is trained using the data stored in the buffer. Additionally, the target network is updated periodically after a certain number of episodes.

```
#compute real q_value and target q-value
q_values=self.policy_network(torch.cat([state,target],dim=-1))
q_value=torch.tensor([0.0]*len(action))
for i in range(len(action)):
    q_value[i]=q_values[i,action[i]]
next_q_value=self.target_network(torch.cat([next_state,target],dim=-1).to(self.device)).max(dim=-1)[0]
target_q_value=reward+self.gamma*next_q_value * (1 - done)
q_value=q_value.to(self.device)
target_q_value=target_q_value.to(self.device)
# grid decrease
loss=self.loss_func(q_value,target_q_value)
self.optimizer.zero_grad()
loss.backward()
self.optimizer.step()
```

To update the policy network, it is necessary to compute both the real q-value and the target q-value. The real q-value is calculated using the current policy network, while the target q-value is computed using the target network according to the formula:
$r + \gamma * max(Target\_Network(s))$ where r represents the reward, γ is the discount factor, and s is the current state.

8.3.2 HER

```
if self.HER is not None and self.my_version==False:
    for j in range(self.env.len):
        for k in range(self.HER):
            new_target = next_state_episode[random.randint(j, self.env.len-1)] # choose 'new target' from 'next state'
            reward,done = self.env.compute_reward(new_target,state_episode[j],action_episode[j]) # compute new reward in new target
            self.replay_buffer.store(
                {'state': state_episode[j], 'target': new_target, 'reward': reward, 'next_state': next_state_episode[j],
                 'action': action_episode[j], 'done': done})
```

After the completion of each episode, the next_state that appeared in that particular episode will be randomly selected as the new target sequence. Subsequently, the reward will be updated and stored in the buffer along with other elements such as the state and the completion status (done).

8.3.3 Curriculum Learning

```
# explore in environment
while not done:
    self.policy_network.train()
    state=obs['state'].to(self.device)
    target=obs['target'].to(self.device)
    action=self.select_action(self.eps,state,target)
    obs,reward,done=self.env.step(action)

 self.replay_buffer.store({'state':state.to(self.device),'target':target.to(self.device),'reward':reward,'next_state':obs['state'].to(self.device),'action':action,'done':done})
if index%5==0 and fixed_length!=0: # add some extra examples to avoid overfitting
    # obs = self.env.reset(0)
    obs = self.env.reset_state(np.random.randint(0,fixed_length))
    done = False
    while not done:
        self.policy_network.train()
        state = obs['state'].to(self.device)
        target = obs['target'].to(self.device)
        action = self.select_action(self.eps, state, target)
        obs, reward, done = self.env.step(action)
        self.replay_buffer.store(
            {'state': state.to(self.device), 'target': target.to(self.device), 'reward': reward,
             'next_state': obs['state'].to(self.device), 'action': action, 'done': done})
```

In each episode, the initial portion of the state sequence is provided. Additionally, every 5 episodes, a random exploration is performed where the length of the given portion varies randomly.

```
# when the acc is larger than 0.9 for 5 continuous epoches, the fixed length will decrease
if len(self.log["current epoch"])>5 and (np.array(self.log["current success rate"][-5:])>0.9).all():
    if fixed_length==0:
        break
    fixed_length-=self.interval
    if fixed_length<0:
        fixed_length=0
    else:
        if fixed_length % self.freeze_interval == 0:
            self.prob -= self.prob_interval
        index = 1
```

Once the model has reached convergence, indicated by achieving a success rate of over 0.9 for 5 consecutive epochs, the fixed length will decrease and certain parameters of the network will be unfrozen.